Estrategia de pruebas

SomnGuard | ADSO 3145555 | Revisión 2026-11-05

Estrategia de pruebas

Estado: En progreso

Fecha de revisión: 2026-11-05

Estrategia de pruebas inicial del proyecto SomnGuard, alineada con la metodología adoptada (Scrum) y la arquitectura definida. Complementa la sección de calidad de `../00-documentation-governance/adopted-methodology.md`.

Objetivo

Garantizar la calidad del sistema desde el inicio, con un enfoque incremental: comenzar con pruebas unitarias en módulos críticos y madurar hacia pruebas de integración, API y E2E conforme avance el desarrollo.

Niveles de prueba

Nivel	Alcance	Herramienta propuesta	Prioridad
Unitarias	Lógica de dominio y casos de uso del backend	JUnit 5 + Mockito (Maven)	Alta
Integración	Persistencia con PostgreSQL, repositorios y migraciones Liquibase (changelogs de `somnguard-db`)	Spring Boot Test + Testcontainers; requiere un daemon Docker accesible	Alta
API	Controladores REST, autenticación, validaciones y respuestas HTTP	Spring Boot Test + MockMvc	Media
Frontend web	Componentes React JS	Jest + React Testing Library	Media
Móvil	Componentes React Native	Jest (React Native Testing Library)	Media
E2E	Flujos críticos completos	Playwright (web)	Baja (sprints posteriores)
Manual	Aceptación de historias de usuario	Checklist + entorno `qa`	Alta

Identificación de casos de prueba

Los casos de prueba se identifican con el prefijo `TC-` (convención en agile-conventions.md) y se registran:

- En la evidencia por historia: _template-test-evidence.md (formato `test-evidence-HU-<REPO>-NNN.md`).
- En el reporte por sprint/release: _template-qa-report.md.
- En la matriz de trazabilidad, vinculando HU -> AC -> TC.

Cada caso de prueba declara: precondición, pasos, resultado esperado y trazabilidad al criterio de aceptación (AC) que verifica.

Criterios de cobertura

- Los módulos críticos (análisis visual en edge, ingesta de eventos, alertas) tendrán prioridad de cobertura.
- Objetivo inicial: al menos un test por caso de uso crítico.
- La cobertura objetivo porcentual se define en un sprint de calidad (ver preguntas abiertas).

Estrategia por módulo

- **security**: pruebas de autenticación, roles y permisos; uso de datos ficticios (nunca datos reales).
- **device-management**: asignación/desasignación, historial y configuración JSONB.
- **telemetry-service**: ingesta de eventos, deduplicación por sincronización offline e integridad de evidencia.
- **monitoring**: generación de notificaciones y transición de estados (emitida, leída).
- **parameterization**: validación de catálogos y reglas de referencia (FK).

Datos de prueba

- Usar siempre datos ficticios (normativa del proyecto).
- Entornos: `local`, `develop`, `qa`, `main`.
- El entorno `qa` es el punto de validación antes de `main`.

Ejecución del API

Ejecutar la suite desde la raíz de `somnguard-api` con Java 21:

```
mvn -B test
```

La mayoría de las pruebas de contexto usan H2 en memoria. `AuthPostgresIntegrationTest` levanta PostgreSQL mediante Testcontainers, por lo que el proceso Maven debe poder comunicarse con Docker. La clave RSA del perfil Spring `test` está aislada en `src/test/resources/keys/test/`; no debe reutilizarse en desarrollo o producción. `test` es un perfil de Spring, no un perfil Maven.

Criterios de aceptación (DoD)

Una historia se considera probada cuando (aplica a la HU):

- [ ] Pruebas unitarias de los casos de uso implementados pasan.
- [ ] Pruebas de integración con base de datos pasan en entorno `develop`.
- [ ] Flujo principal verificado manualmente en entorno `qa`.
- [ ] No se introdujeron regresiones en módulos relacionados.

Relación con CI/CD

Las pruebas se ejecutan en GitHub Actions (ver `../10-devops/ci-cd-strategy.md`): unitarias e integración en cada push, E2E en release. El pipeline debe bloquear el merge si las pruebas fallan.

Requisitos no funcionales

La estrategia verifica los requisitos no funcionales del catálogo (ver `../04-requirements/non-functional.md`): rendimiento (NFR-04) y escalabilidad (NFR-05) se cubren con pruebas de carga en QA; disponibilidad (NFR-08) se monitorea vía SLOs en observabilidad.

Pendientes

- Definir cobertura objetivo.
- Definir herramienta de reportes de cobertura y calidad (ej: JaCoCo).
- Incorporar pruebas automatizadas al DoD formal (según metodología).